Reparto de responsabilidades — Plataforma de Cine, Series y Anime

Oct 9, 2026 · @Wilder

Visión general

El equipo de 4 personas construye primero un MVP: catálogo de películas/series/animes, cartelera real de 1 o 2 cines y reseñas con puntuación. Seguidores, listas personalizadas y sugerencias colaborativas quedan como extras si el tiempo lo permite.

Cada persona es dueña de una parte del backend Django, con sus propias apps y su propia rama de Git. Así se evitan choques en el código y cada quien sabe de qué responde en las revisiones.

Persona	Rol	Apps de Django	Entrega principal
1	Líder técnico y base del proyecto	Raíz del proyecto y configuración	Repositorio y esqueleto funcionando
2	Catálogo	`media`	Modelos, `seed_data` y API del catálogo
3	Cartelera real y scraping	`cines`	Scraper de 1 o 2 cines y API de funciones
4	Usuarios y reseñas	`usuarios`, `social`	Autenticación JWT y reseñas con puntuación

Fuera del MVP por ahora: el frontend (Fase 5 del roadmap), el ERD formal, los seguidores, las listas y las sugerencias de edición con aprobación del administrador.

Reglas de trabajo comunes

Todos trabajan con las mismas reglas desde el primer día, para que el repositorio no se vuelva un caos con 4 personas escribiendo a la vez.

1. **Una rama por funcionalidad.** Nadie escribe directo en `main`. Los nombres siguen el patrón `feature/<area>-<descripcion>`, por ejemplo `feature/catalogo-modelos`, `feature/scraper-cinemark` o `feature/auth-jwt`.
2. **Pull Request obligatorio.** Cada rama se integra a `main` con un PR que revisa al menos otra persona. La Persona 1 hace la revisión final y el merge.
3. **Commits pequeños y claros.** Un mensaje por cambio con sentido, en español o inglés pero siempre el mismo idioma, por ejemplo `Agrega modelo Media` o `Corrige filtro por género`.
4. **Secretos fuera del repositorio.** Claves, contraseñas de la base de datos y tokens van en un archivo `.env` que está en el `.gitignore`. Se sube solo un `.env.example` con los nombres de las variables y valores falsos.
5. **Migraciones sin pisarse.** Si dos personas crean migraciones de la misma app, se avisa antes por el grupo del equipo. Cada persona crea migraciones solo de sus propias apps.
6. **Entorno local igual para todos.** Python con entorno virtual (`venv`), dependencias en `requirements.txt` y PostgreSQL local. El README de la Persona 1 explica los pasos exactos.
7. **Una reunión corta de sincronización por semana** de 15 a 20 minutos: qué hizo cada uno, qué lo bloquea y qué sigue.

Persona 1: Líder técnico y base del proyecto

Es la persona que destraba a las demás: nadie puede escribir modelos ni endpoints hasta que el esqueleto del proyecto exista. Además coordina la integración del código de todos.

Qué hace exactamente

1. Crea el repositorio en GitHub, invita a las otras 3 personas y protege la rama `main` para que solo se modifique con Pull Requests.
2. Crea el proyecto Django con las 4 apps vacías: `media`, `cines`, `social` y `usuarios`.
3. Instala y configura Django REST Framework, `django-cors-headers`, `djangorestframework-simplejwt`, `python-decouple` (o `django-environ`) y el driver `psycopg2-binary`.
4. Divide las settings en una base común y una configuración local, leyendo todos los secretos desde `.env`.
5. Configura PostgreSQL: crea la base de datos de desarrollo y documenta cómo hacerlo en cada sistema operativo.

6. Escribe el `.gitignore`, el `.env.example`, el `requirements.txt` y un README con los pasos para levantar el proyecto en local en menos de 15 minutos.
7. Define la estructura de carpetas de cada app para que todas sean iguales: `models.py`, `serializers.py`, `views.py`, `urls.py`, `services.py` (capa de servicio) y `tests/`.
8. Prepara las URLs base de la API bajo el prefijo `/api/` y deja un endpoint de prueba (`/api/health/`) que confirme que todo está conectado.
9. Revisa los Pull Requests del equipo, resuelve conflictos de merge y mantiene `main` siempre funcionando.
10. Más adelante: pruebas de integración, revisión de rendimiento de queries (`select_related`, índices) y documentación de arquitectura (Fases 6 y 7).

Entregables

- Repositorio con el esqueleto subido a `main` en los primeros 1 o 2 días.
- README, `.env.example` y `requirements.txt` completos.
- Un mensaje al equipo avisando que ya pueden clonar y empezar sus ramas.

Terminado cuando: las otras 3 personas clonan el repo, siguen el README, ejecutan `python manage.py runserver` y ven responder `/api/health/` sin ayuda adicional.

Persona 2: Catálogo (app `media`)

Es quien se encarga del corazón de los datos: las películas, series y animes. Casi todo el resto del sistema depende de sus modelos (las funciones de cine y las reseñas apuntan a `Media`), así que sus modelos deben estar listos pronto, aunque sean una primera versión.

Qué hace exactamente

1. Crea los modelos de la app `media`: `Media` (con un campo `tipo` que distingue película, serie o anime), `Genero`, `Persona` (actores, directores) y `Credito` (qué persona participó en qué media y con qué rol).
2. Incluye en `Media` los campos mínimos del MVP: título, título original, sinopsis, fecha de estreno, duración, póster (URL), tráiler (URL), tipo y estado (en cartelera, próximo estreno, fuera de cartelera).
3. Genera las migraciones y las sube cuanto antes, para que las Personas 3 y 4 puedan relacionar sus modelos con `Media`.
4. Escribe el comando `seed_data` (`python manage.py seed_data`) que carga al menos 30 títulos reales con géneros, personas y créditos. Este comando es lo que permite al resto del equipo trabajar sin depender del scraper. Debe poder ejecutarse varias veces

sin duplicar datos.

5. Crea los serializers y las vistas de solo lectura: listado de media, detalle de una ficha (con reparto y géneros) y listado de géneros.
6. Implementa la búsqueda y los filtros por texto, género, tipo, año y estado (por ejemplo `/api/media/?genero=accion&tipo=pelicula&anio=2025`), con paginación.
7. Configura el Django Admin para `Media`, `Genero`, `Persona` y `Credito` (listados, búsqueda y filtros), de modo que el administrador pueda hacer el CRUD desde ahí.
8. Define los permisos: lectura pública para visitantes, y escritura solo para administradores.
9. Escribe pruebas para los filtros y el detalle de la ficha.

Entregables

- Modelos y migraciones de `media` en `main` durante la primera semana.
- Comando `seed_data` funcionando.
- Endpoints `/api/media/`, `/api/media/<id>/` y `/api/generos/`.
- Admin configurado.

Terminado cuando: con la base de datos vacía, ejecutar las migraciones y `seed_data` deja un catálogo navegable en la API y editable en el admin, y los filtros devuelven resultados correctos.

Persona 3: Cartelera real y scraping (app `cines`)

Es quien resuelve la parte más riesgosa del proyecto: conseguir los horarios reales de los cines de la ciudad. Su trabajo empieza con investigación, no con código del proyecto, así que puede avanzar desde el primer día sin esperar el esqueleto.

Qué hace exactamente

1. **Reconocimiento de los sitios** de Cinemark, Cinépolis, Royal Films y Cineland. Con las herramientas de desarrollo del navegador (pestaña Network), busca si cada sitio carga la cartelera desde un endpoint JSON interno, que es mucho más fácil y estable de consultar que parsear HTML.
2. **Informe corto de hallazgos** para el equipo: por cada cine, qué URL usa, qué datos entrega (película, sala, hora, formato), si requiere Playwright y qué tan difícil se ve. Con eso el equipo decide cuáles son los 1 o 2 cines del MVP.

3. Crea los modelos de la app `cines`: `Cine` (nombre, ciudad, dirección, sitio web) y `Funcion` (relación con `Media` y con `Cine`, fecha y hora, sala, formato como 2D, 3D o subtitulada, fuente del scraping y fecha de última actualización).
4. Escribe el **primer scraper** (por ejemplo `scrapers/cinemark.py`) con Playwright o con peticiones directas al endpoint JSON, según lo que encontró.
5. Escribe la **normalización**: cada scraper entrega los datos en un formato común (título, cine, fecha, hora, formato) antes de guardarlos, para no duplicar funciones.
6. Escribe el **matching de títulos** en `services.py`: asocia el título que viene del cine con el `Media` del catálogo, normalizando mayúsculas, tildes y subtítulos. Si no encuentra coincidencia, lo registra en una lista de pendientes para revisión manual en lugar de inventar un `Media`.
7. Crea un comando de gestión (`python manage.py scrape_cartelera --cine cinemark`) para ejecutar el scraper a mano.
8. Crea los endpoints de lectura: `/api/cines/`, y `/api/media/<id>/funciones/` con los horarios de una película, filtrables por cine y por fecha. La respuesta incluye cuándo se actualizó la información por última vez.
9. Más adelante (Fase 3): automatiza con Celery y Celery Beat para ejecutar los scrapers 1 o 2 veces al día.
10. **Scraping responsable:** peticiones espaciadas y moderadas, con un `User-Agent` claro, sin sobrecargar los sitios.

Entregables

- Informe de reconocimiento de los 4 cines, durante la primera semana.
- Modelos `Cine` y `Funcion` con migraciones.
- Un scraper funcionando de punta a punta, con normalización y matching.
- Endpoints de cines y de funciones por película.

Terminado cuando: al ejecutar el comando del scraper, las funciones reales del cine elegido aparecen en la base de datos, asociadas a películas del catálogo y sin duplicados al correrlo dos veces seguidas.

Persona 4: Usuarios y reseñas (apps `usuarios` y `social`)

Es quien construye la parte "red social" del proyecto en su versión mínima: que alguien pueda registrarse, iniciar sesión y puntuar o reseñar una película. También define los permisos que usa todo el sistema (visitante, usuario registrado y administrador).

Qué hace exactamente

1. Crea el modelo de usuario en la app `usuarios`. Recomendación: un modelo propio basado en `AbstractUser` desde el inicio (con campos como `avatar` y `biografia`), porque cambiarlo después de las primeras migraciones es muy doloroso. Esta decisión se avisa a la Persona 1 antes de escribirla, porque afecta `AUTH_USER_MODEL` en las settings.
2. Implementa el registro, el login y el refresco de token con JWT (`djangorestframework-simplejwt`): `/api/auth/registro/`, `/api/auth/login/` y `/api/auth/refresh/`.
3. Crea el endpoint del perfil propio (`/api/usuarios/me/`) para ver y editar los datos del usuario autenticado.
4. Define los **permisos reutilizables** en un módulo común: `EsAdministrador`, `EsAutorOLecturaPublica` (el usuario edita solo lo suyo) y lectura abierta para visitantes. El resto del equipo los importa en sus vistas.
5. Crea el modelo `Resena` en la app `social`: relación con `Usuario` y con `Media`, puntuación (por ejemplo de 1 a 10 o de 0.5 a 5, a decidir con el equipo), texto opcional y fecha. Un usuario puede tener solo una reseña por título.
6. Crea los endpoints de reseñas: crear, editar y borrar la propia, y listar las reseñas de un título (`/api/media/<id>/resenas/`), con paginación.
7. Escribe en `services.py` el **cálculo del promedio de puntuación** de la comunidad y la cantidad de votos por título, y se expone para que la ficha del catálogo lo muestre. Si el cálculo se vuelve lento, se guarda en un campo desnormalizado.
8. Permite que el administrador borre o oculte reseñas inapropiadas desde el Django Admin.
9. Escribe pruebas de registro, login, permisos (que un usuario no pueda editar la reseña de otro) y promedio.
10. Cuando el backend del MVP esté estable, empieza los wireframes y la identidad visual del frontend (Fase 5).

Entregables

- Modelo de usuario y autenticación JWT en `main` durante la primera semana, porque las otras personas necesitan los permisos.
- Módulo de permisos reutilizable.
- Modelo y endpoints de reseñas, con promedio de puntuación.

Terminado cuando: una persona nueva puede registrarse, iniciar sesión, dejar una reseña en una película del catálogo, ver el promedio actualizado, y no puede modificar reseñas ajenas.

Puntos de integración y acuerdos previos

Hay 5 puntos donde el trabajo de una persona toca el de otra. Cada uno se acuerda por el grupo del equipo antes de escribir el código, para no tener que rehacerlo después.

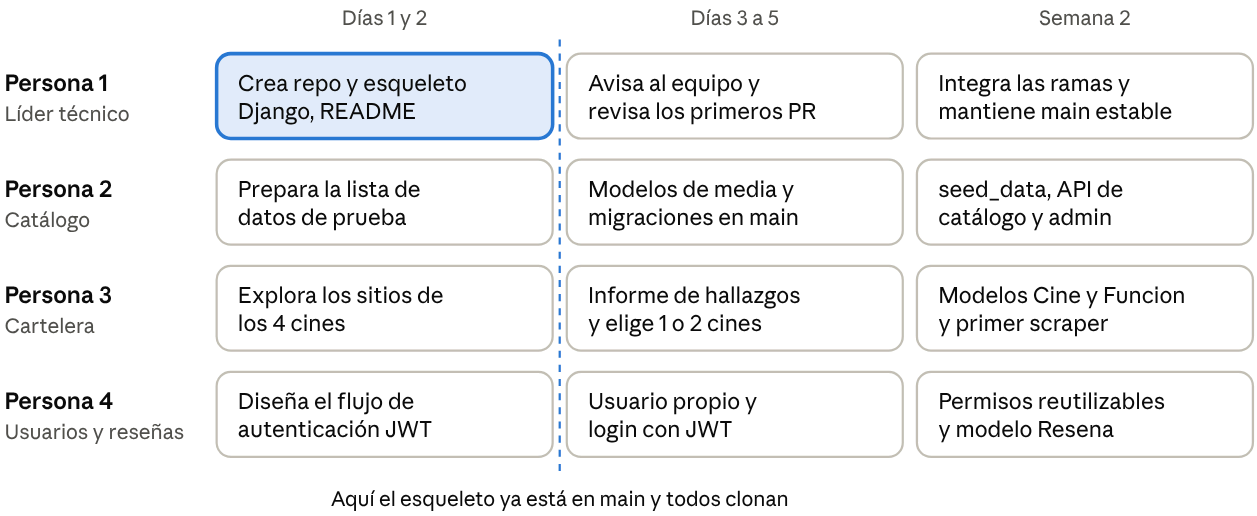
Punto de integración	Quiénes	Qué hay que acordar	Cuándo
Modelo de usuario personalizado	Personas 1 y 4	Usar `AbstractUser` propio y fijar `AUTH_USER_MODEL` antes de la primera migración	Día 1 o 2, antes de migrar
Modelo `Media`	Personas 2, 3 y 4	Nombre exacto del modelo y de su clave primaria, para relacionarlo desde `Funcion` y `Resena`	Primeros 3 días
`Funcion` con `Media` y `Cine`	Personas 2 y 3	Cómo se marca un título como "en cartelera" y qué pasa cuando el scraper no encuentra coincidencia	Semana 2
Promedio de puntuación en la ficha	Personas 2 y 4	Si el serializer de `Media` incluye el promedio, y quién escribe esa parte	Semana 2
Permisos compartidos	Personas 1, 2, 3 y 4	La Persona 4 publica los permisos y todos los usan, en vez de escribir los suyos	Semana 1

Una regla para todos los acuerdos: lo que se decide se escribe en el README o en un archivo `DECISIONES.md` del repositorio, con fecha y quién participó. Así nadie depende de recordar lo que se dijo en el chat.

Plan de las primeras 2 semanas

El esqueleto sale en los días 1 y 2, y desde ahí las 4 personas avanzan en paralelo, cada una en su propia rama.

Con el esqueleto en main al día 2, las cuatro personas trabajan en paralelo



plan de 2 semanas · 4 personas en paralelo

Mientras la Persona 1 sube el esqueleto, las Personas 2, 3 y 4 hacen trabajo que no depende de él: datos de prueba, reconocimiento de los cines y diseño de la autenticación. Los modelos de `media` y el usuario propio quedan en `main` antes de que termine la primera semana, porque de ellos dependen las funciones de cine y las reseñas.

Después de estas 2 semanas siguen las Fases 2 y 3 del roadmap: se completa la API del catálogo, el scraper pasa a automatizarse con Celery y se cierra el MVP con las reseñas y el promedio de puntuación. El reparto se revisa en la reunión semanal según el ritmo real del equipo.